

FFT (fast fourier transform)

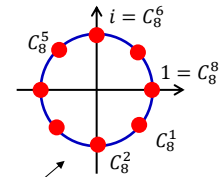
Reminder of the DFT:

$$f(k \cdot \Delta\omega) \approx \sum_{n=0}^{N-1} F(n\Delta t) e^{-ik\Delta\omega n\Delta t} \cdot \Delta t, \quad \Delta\omega = \frac{2\pi}{N\Delta t}$$

Δt is then dropped from the further calculations. (It can be done, if the amplitude is not important, just the spectral decomposition. Later, in noise measurements, it is not possible, as the noise is a physical quantity, and its true amplitude is important.)

$$f(k) = \sum_{n=0}^{N-1} F(n) e^{-i2\pi nk/N} = \sum_{n=0}^{N-1} F(n) (C_N)^{kn}, \quad k = 0..N-1; N = 2^p; C_N = e^{-i2\pi/N}$$

The number of datapoints is a power of 2, we will see later, why.



These are complex numbers spaced equidistantly around the complex unit circle

Number of operations : 1 k point: N multiplication & N-1 addition, ~N „operations“. If we evaluate N k points: altogether ~N² operations. Now we ignore the previously discussed symmetry of the DFT, which would half the k points, as we will need all the N k points later for the FFT.

Trick: we calculate the DFT separately for the even (n=2r) and odd (n=2r+1) points:

$$f(k) = \sum_{r=0}^{N/2-1} F(2r) C_N^{2rk} + \sum_{r=0}^{N/2-1} F(2r+1) C_N^{(2r+1)k} = \sum_{r=0}^{N/2-1} F(2r) (C_N^2)^{kr} + C_N^k \sum_{r=0}^{N/2-1} F(2r+1) (C_N^2)^{kr}$$

$$f(k) = \underbrace{\sum_{r=0}^{N/2-1} F(2r) C_N^{kr}}_{f_{\text{even}}(k)} + C_N^k \underbrace{\sum_{r=0}^{N/2-1} F(2r+1) C_N^{kr}}_{f_{\text{odd}}(k)}$$

$$C_N^2 = e^{-i4\pi/N} = C_{N/2}$$

17

FFT (fast fourier transform)

$f_{\text{even}}(k)$ and $f_{\text{odd}}(k)$ are each an N/2-point DFT, and it is enough to evaluate the k to 0..N/2-1, since e.g:

$$f_{\text{even}}(k - N/2) = \sum_{r=0}^{N/2-1} F(2r) C_{N/2}^{(k-N/2)r} = \sum_{r=0}^{N/2-1} F(2r) C_{N/2}^{kr} \underbrace{C_{N/2}^{-rN/2}}_1 = f_{\text{even}}(k) \quad \text{and similarly, for } f_{\text{odd}}(k)$$

That is, computing $f_{\text{even}}(k)$ and $f_{\text{odd}}(k)$ is (N/2)² + (N/2)² operations, and computing f(k) from the two is further N operations (see the C_N^k multiplication of f_{odd}), i.e. N²/2+N operations in total instead of the original N²!!

Since we have N=2^p data points, the above two N/2 point DFTs can be further divided into four N/4 point DFTs. Finally, the original N-point DFT can be reduced to N=2^p single-point DFTs. From these we can calculate f(k) with a small number of operations:

Elementary DFT number of points	To calculate the full DFT, i.e. f(k) number of operations:
N	N ²
$\frac{N}{2}$	$2\left(\frac{N}{2}\right)^2 + N = \frac{N^2}{2} + N$
$\frac{N}{4}$	$2\left(2\left(\frac{N}{4}\right)^2 + \frac{N}{2}\right) + N = \frac{N^2}{4} + 2N$
$\frac{N}{8}$	$2\left(2\left(2\left(\frac{N}{8}\right)^2 + \frac{N}{4}\right) + \frac{N}{2}\right) + N = \frac{N^2}{8} + 3N$
...	...
$\frac{N}{2^p} = 1$	$\frac{N^2}{2^p} + pN = \frac{N^2}{N} + N \log_2 N \approx N \log_2 N$

Example: N=10⁹ datapoints, assume that a single operation takes 1ns time:

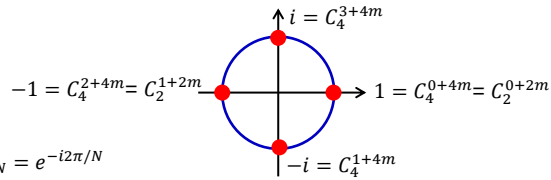
„brute force“ DFT calculation (N² operations):
10¹⁸ ns ~ 31 years

FFT algorithm: (N*log₂N operations):
30 * 10⁹ ns ~ 30 seconds

18

Example: 4 point FFT

$$f_k = \sum_{n=0}^{N-1} F_n C_N^{kn}, k = 0..N-1; N = 2^p; C_N = e^{-i2\pi/N}$$



$$\begin{aligned} f_0 &= F_0 C_4^0 + F_1 C_4^0 + F_2 C_4^0 + F_3 C_4^0 = F_0 C_2^0 + F_2 C_2^0 + C_4^0 \cdot [F_1 C_2^0 + F_3 C_2^0] \\ f_1 &= F_0 C_4^1 + F_1 C_4^1 + F_2 C_4^1 + F_3 C_4^1 = F_0 C_2^1 + F_2 C_2^1 + C_4^1 \cdot [F_1 C_2^1 + F_3 C_2^1] \\ f_2 &= F_0 C_4^2 + F_1 C_4^2 + F_2 C_4^2 + F_3 C_4^2 = F_0 C_2^0 + F_2 C_2^0 + C_4^2 \cdot [F_1 C_2^0 + F_3 C_2^0] \\ f_3 &= F_0 C_4^3 + F_1 C_4^3 + F_2 C_4^3 + F_3 C_4^3 = F_0 C_2^1 + F_2 C_2^1 + C_4^3 \cdot [F_1 C_2^1 + F_3 C_2^1] \end{aligned}$$

16 operations

 $f_{\text{even}}(k)$
4 operations

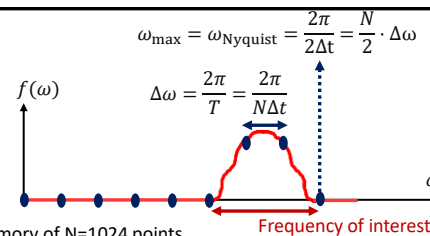
 $f_{\text{odd}}(k)$
4 operations

That is, the 16-operation 4-point DFT has been reduced to 12 operations by reducing the even and odd points back to a 2-2-point DFT (see FFT algorithm). However, if we are only interested in the last two frequency points (f_3, f_4), the direct count is 8 operations, but the FFT count saves only 2 operations (C_4^0 and C_4^1 multiplication), i.e. 10 operations are needed. This shows that the FFT algorithm is only effective if all frequency points from zero frequency to f_{max} are calculated. In other words, even if we want to evaluate only a limited frequency range of the spectrum, the FFT algorithm takes almost as many operations as if we evaluate the whole spectrum.

19

FFT spectrum analyzer

The analyzed signal is sampled at Δt sampling intervals, and the spectrum is obtained by calculating the Fourier transform of the signal using the FFT algorithm. Here the maximum frequency in the spectrum is the Nyquist frequency, i.e. for high bandwidth a fast sampling is needed, whereas the good frequency resolution is granted by a long measurement time.



Example: we have an (old) spectrum analyzer with a total memory of $N=1024$ points

- If we sample for $T=1\text{ms}$ ($\Delta t=1\text{ms}/1024 \sim 1\mu\text{s}$ time resolution) we can resolve the FFT spectrum to components with frequency $f_k=k/T, k=0..N/2$, i.e. our frequency resolution is $\Delta f=1/1\text{ms}=1\text{kHz}$. With this resolution we can see the signal from DC to $f_{\text{max}}=512\text{kHz}$ Nyquist frequency.
- If we sample for $T=1\text{s}$ ($\Delta t=1\text{s}/1024 \sim 1\text{ms}$ time resolution) then in the FFT spectrum we have a frequency resolution of $\Delta f=1\text{Hz}$, with this resolution we see the signal from DC to $f_{\text{max}}=512\text{Hz}$.

That is: either we see the signal with good frequency resolution but only down to low frequency, or with poor frequency resolution but up to high frequency

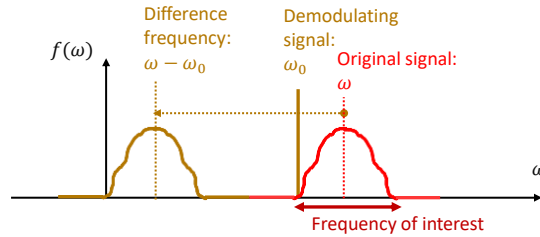
In the previous example, the limiting factor was the small memory of our (old) spectrum analyzer, because we could only record 513 discrete frequency points. Consider a new spectrum analyzer with a **memory of 1Msample** (10^6). With this, and a **sampling time of 1s**, we can see the signal from **DC to 0.5MHz** with a **resolution of 1Hz** in the frequency domain, i.e. we have good frequency resolution at high frequencies!

Problem: the FFT calculation is slow because of the many points. Moreover, when computing the FFT, we cannot compute only the **frequency components of interest** to us, since the FFT is only fast if we compute all the points at once, and **with fewer ω points the number of operations does not scale down with the number of points.**

20

Heterodyne spectrum analysis

What if we want to see the signal not from DC but only for the **frequency range of interest**, but with good frequency resolution, and also avoiding the computation intensive FFT algorithm for millions of points?



Trick: Before calculating the Fourier transform, multiply the signal by a harmonic signal with circular frequency ω_0 .

The Fourier transform of the multiplication:

$$\tilde{f}_W(\omega) = \int_{-\infty}^{\infty} dt W(t) F(t) e^{i\omega_0 t} e^{-i\omega t} = f_W(\omega - \omega_0), \text{ where } f_W(\omega) = \int_{-\infty}^{\infty} dt W(t) F(t) e^{-i\omega t}$$

That is, the frequency component ω of the original signal appears in the Fourier transform of the multiplication shifted by ω_0 ! The frequency resolution is still determined by the sampling length T : $\Delta\omega \sim 2\pi/T$.

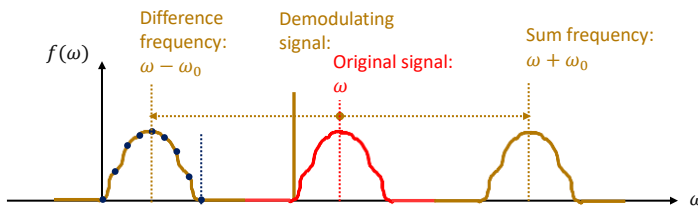
In measurement, we can only multiply by a real signal, so let us calculate the Fourier transform of a signal multiplied by a cosine signal:

$$\tilde{f}_W(\omega) = \int_{-\infty}^{\infty} dt W(t) F(t) \frac{\cos(\omega_0 t)}{e^{i\omega_0 t} + e^{-i\omega_0 t}} e^{-i\omega t} = \frac{1}{2} f_W(\omega - \omega_0) + \frac{1}{2} f_W(\omega + \omega_0)$$

That is, the difference and sum frequencies are also appearing. If the high-frequency component (sum frequency) in the multiplicative signal is suppressed by a low-pass filter and the FFT is applied to the resulting signal, then, e.g. with a measurement time of 1s, a memory of 1024 points and $f_0=500\text{kHz}$ "multiplication signal", the difference frequency signal $f_w(\omega - \omega_0)$ is displayed with a resolution of 1Hz from DC to 512Hz, i.e. the spectrum $f_w(\omega)$ of the original signal is recorded with a resolution of 1Hz in the range 500000-500512Hz! This is called **heterodyne detection**. As the signal of interest is shifted close to the DC domain, we can use slower sampling with less datapoints, which saves computation time, and we don't need fast A/D converters.

21

Hybrid spectrum analyzer



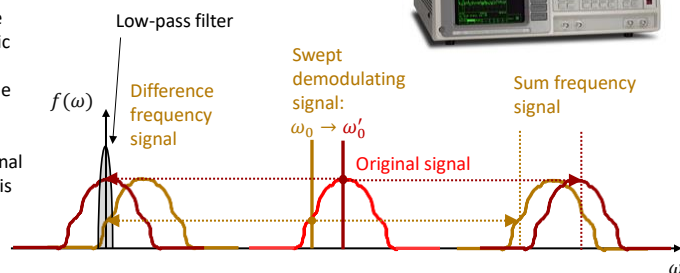
$$\Delta\omega = \frac{2\pi}{T} = \frac{2\pi}{N\Delta t}$$

$$\omega_{\max} = \frac{N}{2} \cdot \Delta\omega$$

The signal is down-converted with the fixed ω_0 frequency multiplying signal corresponding to the frequency window of interest, and the FFT algorithm is applied to the resulting difference frequency signal. Here, a slower sampling is enough to calculate the FFT of the difference frequency signal. The sum frequency signal and the leftovers of the original frequency signal in the product spectrum are suppressed by low-pass filters.

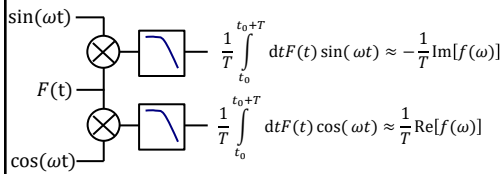
Swept-tuned spectrum analyzer

Using heterodyne detection, the signal is multiplied by a harmonic signal of frequency ω_0 , the DC component is measured from the multiplied signal by a low-pass filter (this gives the Fourier component of the measured signal of frequency ω_0). The spectrum is determined by tuning the frequency of the multiplying signal.



22

Swept-tuned spectrum analyzer in more detail



The input signal of the spectrum analyzer ($F(t)$) is multiplied by a signal of circular frequency ω ($\cos(\omega t)$) and its 90 degree shift ($\sin(\omega t)$). The multiplied signals are passed through low-pass filters. A low-pass filter is equivalent to integrating the (multiplied) signal over a certain time T . It can be seen that the two outputs are proportional to the real and the imaginary parts of the Fourier transform of the signal $F(t)$ at frequency ω . By varying the circular frequency ω , we can map the full Fourier spectrum.

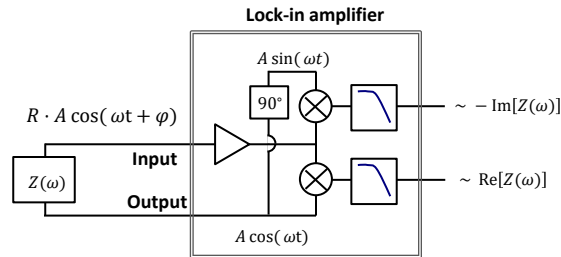
Lock-in amplifier

Practically the same thing is done by the so-called lock-in amplifier. The only difference is that $F(t)$ is not an arbitrary external signal, but a physical system is driven by the $\cos(\omega t)$ signal output by the lock-in and the response is measured at the input of the lock-in. For a linear system, the response function is a multiplication by a complex number ($Z(\omega)$), which can be described by an amplitude (R) and a phase shift (ϕ).

The lock in, after amplification, multiplies the signal at its input by the drive signal ($\sim \cos(\omega t)$) and its 90 degree shift ($\sim \sin(\omega t)$), passing the multiplied signals through a low-pass filter (for a digital lock in amplifier, this is effectively integration). The resulting signals are the real and the imaginary parts of the complex response function $Z(\omega)$. This method effectively filters out any interfering signal at a frequency other than that of the drive. This is called **phase-sensitive detection**.

$$\text{Drive: } \text{Re}[A \cdot e^{i\omega t}] = A \cdot \cos(\omega t)$$

$$\text{Response: } \text{Re}[Z(\omega) \cdot A \cdot e^{i\omega t}] = \text{Re}[Z(\omega)] \cdot A \cdot \cos(\omega t) - \text{Im}[Z(\omega)] \cdot A \cdot \sin(\omega t)$$



23

Phase locked loop, PLL

Goal: to synchronize the output of the PLL to the input signal (this is also used in a lock-in, see next slide)

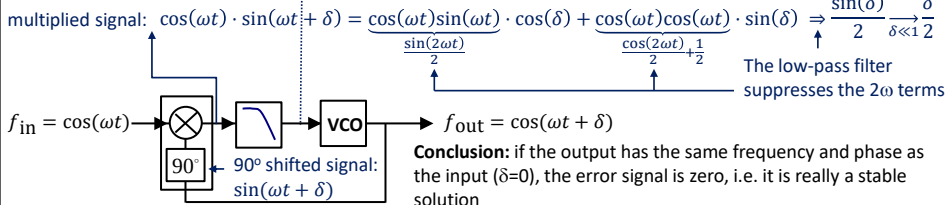
Phase detector: generates an error signal proportional to the phase difference between the output and input signal

Low-pass filter: filters the sum frequency from the multiplied signal

Voltage controlled oscillator: changes the output frequency with a speed proportional to the input voltage (i.e. the error signal), $df/dt \sim V_{in}$. This is done with a PI regulator (see temperature control in one of the first lectures) using a setpoint, where the error signal is zero.

Fixpoint: if the output and input signals are of the same frequency and phase, the phase detector gives a zero error signal, so the VCO does not change frequency. The output signal is then synchronized to the input signal. If the input signal is "noisy" around a given frequency, it will be "cleaned up" at the output.

Example:



If, however, the output has the same frequency but different phase (nonzero constant δ), or moreover, the output is somewhat detuned in frequency (consider $\delta = \Delta\omega \cdot t + \delta_0$), the nonzero error signal is compensated by the PI loop of the PLL and the VCO will finally find the stable solution, where the output is "locked" to the input.

24